Application Observability

Day 13 | AWS / AZURE / GCP + OPEN SOURCE

Use this guide to investigate a specific failure, apply a cause-specific change and verify recovery. Start with evidence from the affected workload. The examples require adaptation to your environment.

How the failure happens

Useful observability connects a user request with the work performed across its dependencies. Metrics reveal a distribution or trend, traces identify request-path segments, and structured logs provide events that can be correlated with trace context.

The Scenario Health endpoints return 200, but a checkout waits on a slow dependency.

What You Need Correlate request latency, trace spans and structured logs.

Open-Source Stack OpenTelemetry, Prometheus, Grafana, Loki, and Tempo or Jaeger. Choose components deliberately.

Before you start

An instrumentable application pair; propagated trace context; a local Collector or approved backend; known sampling and data-handling policy; one reproducible slow request.

Diagnostic checks

```
# Find a slow request trace ID.
# Follow client and server spans.
# Correlate dependency time with logs/metrics.
```

Missing Spans?: Instrument both services and propagate trace context across HTTP and messaging boundaries.

Check Collection: Review sampling, exporter errors, dropped spans and collector connectivity.

Apply the cause specific fix

Choose Your Backend

Aws: CloudWatch Application Signals, logs and tracing. Instrument the actual application.

Azure: Azure Monitor, Application Insights and Log Analytics.

Gcp: Cloud Monitoring, Cloud Logging and Cloud Trace. Use compatible instrumentation/export.

Fix The Confirmed Cause

```
# Local lab Collector config: traces to debug
receivers:
  otlp:
    protocols:
      http: {endpoint: "127.0.0.1:4318"}
exporters:
  debug: {}
service:
  pipelines:
    traces:
      receivers: [otlp]
      exporters: [debug]
```

Prove The Pipeline Locally: Instrument a local app and export OTLP/HTTP traces to this Collector.

Then Use Your Backend: Replace debug export with a supported authenticated backend pipeline. This is not durable trace storage.

Validate the result

```
# Send a known request across both services.
# Find its trace and expected dependency spans.
# Introduce a controlled delay in a lab.
```

Expected: The trace identifies where the injected delay occurs.

Control Data: Redact sensitive attributes and set sampling, retention and cardinality limits.

Practice exercise

Instrument two local services. Introduce a known dependency delay and locate the responsible span. Remove one propagation header to see how the trace breaks, then restore it.

Implementation notes

Collector config is a local lab, bound to loopback. Run app and Collector on the same host/network namespace or adapt networking deliberately. Use the correct OTLP protocol and endpoint. The debug exporter can expose data and is not a production backend. Production requires auth/TLS, resource limits, batching, retry/queue design and a supported exporter/distribution. OpenTelemetry instruments and collects; it does not itself replace telemetry storage. Cloud features differ by runtime and onboarding path.

Interview question

Can OpenTelemetry replace the telemetry database? It provides instrumentation and collection components; durable storage and investigation require suitable backends.

Record what changed

- The symptom and affected user operation.
- Evidence supporting the root cause and alternatives ruled out.
- The changed configuration and the version or identity involved.
- Positive and negative tests, plus any remaining risks.

Example status

Examples are educational templates or fragments. They were not deployed against a live cloud account or production cluster. A successful syntax check is not proof of runtime compatibility. Test adapted configurations in an isolated environment first.

Official references

Official documentation 1

Official documentation 2

Official documentation 3

Official documentation 4